AI/ML Graduate Course — Project Idea Bank

Prepared for: Graduate-level AI course • Team size: 3 • Timeline: 2 months (starting Sept 21, 2026, target completion ~Nov 21, 2026)

Original Prompt

Role: You are an expert academic advisor for AIML courses.

Instruction: Generate 5 industry grade AI project ideas, each for application style projects and research style projects (more preference to Agentic AI).

Context: This is a graded academic project for a graduate level AI course, having a major chunk in the entire subject's grade. The team size for this projects is 3 members. Consider the total project development timeline of 2 months starting from today.

Constraint: Make sure the project has work in which maximum amount of work done on the AI component can be displayed, and there should be low overhead of other components such as UI, DB, Backend, etc. (Minimal is fine so that the team can present a live project to showcase during presentation, eg a UI or a terminal interface). The intention behind this is to not worry about the standard app development part and rather focusing on the significant AI development part.

Example: No example needed, you can craft it yourself

Note: Generate the output in a simple pdf, black font on white background. At the start of the pdf, include the above given prompt

---

This document lists 10 project ideas — 5 **application-style** and 5 **research-style** — weighted toward **Agentic AI** as requested. Every idea is scoped so that at least 80–90% of team effort goes into the AI/ML core (agent design, model behavior, evaluation, algorithms), with only a thin terminal or single-page interface needed for the live demo. Each entry includes the AI core, why it is "industry grade," a suggested minimal interface, an 8-week timeline, and a 3-person role split.

Part A — Application-Style Projects

These projects emphasize building a working, demonstrable AI system. The "application" is mostly the agent or model pipeline itself; supporting UI is deliberately kept to a CLI, notebook, or a single Streamlit page.

Project 1: Autonomous Multi-Agent Research Assistant (Agentic AI)

Category: Application — Agentic AI

AI Core / What you're building

A team of cooperating LLM agents (Planner, Researcher, Critic, Writer) that takes a research question, autonomously decomposes it into sub-tasks, searches/reads sources via tools, cross-checks claims, iteratively critiques its own draft (reflection loop), and produces a cited report — without a human in the loop after the initial query.

Why this is industry-grade

This mirrors real products like Perplexity/OpenAI "deep research" agents. It showcases agent orchestration, tool-use, self-critique/reflection, and retrieval grounding — all core, resume-worthy agentic-AI skills that recruiters and industry panels immediately recognize.

Key techniques & tools

•Agent orchestration framework: LangGraph, CrewAI, or AutoGen
•Tool use: web search API, PDF/HTML parsers, calculator/code tool
•Reflection / self-critique loop (Reflexion-style) to catch hallucinated claims
•RAG with a vector store (FAISS/Chroma) for source grounding and citation tracking
•Evaluation harness: faithfulness/citation-accuracy scoring on a held-out question set

Minimal interface (for the demo)

A terminal CLI (or single Jupyter notebook) where the user types a research question and watches the agents' intermediate reasoning/tool calls stream live, ending in a Markdown report with citations.

8-week timeline

Weeks	Milestone
Wk 1-2	Define agent roles, set up orchestration framework, integrate one tool (search).
Wk 3-4	Add RAG grounding + citation tracking; build first end-to-end pipeline.
Wk 5-6	Add Critic agent + reflection loop; build evaluation set (20-30 questions) and metrics.
Wk 7	Run ablations (with/without reflection, with/without RAG); tune prompts.
Wk 8	Polish CLI demo, write report, record failure-case analysis.

Team split (3 members)

- Member 1: Planner/Researcher agent logic + tool integration
- Member 2: Critic/reflection loop + RAG/citation grounding
- Member 3: Evaluation harness, metrics, ablation studies, and demo CLI

Demo deliverable

Live query typed into the terminal → visible multi-agent trace → final cited report, plus a slide showing accuracy/faithfulness numbers before and after adding reflection.

Project 2: Agentic Code-Review & Auto-Refactor Swarm

Category: Application — Agentic AI

AI Core / What you're building

A multi-agent system that ingests a GitHub repo/PR diff, and autonomously: (a) reviews the diff for bugs, security issues, and style violations, (b) writes/executes unit tests in a sandbox to verify behavior, and (c) proposes and applies safe refactors, iterating until tests pass.

Why this is industry-grade

Directly mirrors industry tools (GitHub Copilot Workspace, Cursor's agents, Devin-style coding agents). Demonstrates tool-calling with code execution, verification loops, and autonomous multi-step planning — a top hiring signal for agentic-AI roles.

Key techniques & tools

•Code-capable LLM + sandboxed execution (Docker or restricted subprocess) for running tests
•Static analysis tools (pylint/bandit) invoked as agent tools
•ReAct-style loop: propose patch → run tests → observe → retry
•Repo/AST parsing (tree-sitter) for structured code understanding
•Guardrails to stop infinite loops (max iterations, diff-size limits)

Minimal interface (for the demo)

A CLI command like `python review.py --repo --pr ` that prints the agent's step-by-step reasoning and outputs a patch file plus a pass/fail test report.

8-week timeline

Weeks	Milestone
Wk 1-2	Sandbox setup, static analysis tool wrapper, basic single-pass reviewer agent.
Wk 3-4	Add test-writing agent + execution loop; get end-to-end patch-and-verify working.
Wk 5-6	Add iterative refactor loop with retry/rollback; curate a benchmark set of 15-20 buggy repos.
Wk 7	Evaluate patch success rate, false-positive rate on review comments; tune loop limits.
Wk 8	Finalize CLI, write report comparing single-pass vs. iterative agentic performance.

Team split (3 members)

- Member 1: Sandbox execution + test-writing agent
- Member 2: Review/refactor agent + static analysis integration
- Member 3: Benchmark curation, evaluation metrics, CLI polish

Demo deliverable

Feed a real (seeded-with-bugs) repo live, show the agent's reasoning trace, and end with a green test suite plus a before/after diff.

Project 3: Autonomous Data-Pipeline Anomaly & Root-Cause Agent

Category: Application — Agentic AI

AI Core / What you're building

An agent that watches structured logs/metrics from a (simulated) data pipeline, detects anomalies, autonomously investigates by querying logs/schemas/metrics via tools, forms a root-cause hypothesis, and proposes/executes a remediation script.

Why this is industry-grade

Mirrors real AIOps/observability agents (Datadog Bits AI, PagerDuty agents). Showcases anomaly detection combined with agentic reasoning over structured + unstructured signals — a growing enterprise-AI niche.

Key techniques & tools

•Statistical/ML anomaly detection (Isolation Forest, Prophet, or a simple autoencoder) on synthetic metrics
•LLM agent with tools: log-query tool, schema-lookup tool, metric-query tool
•Causal/hypothesis-ranking step (LLM-scored candidate causes vs. ground truth)
•Simulated pipeline with injected failure modes (schema drift, null spikes, latency spikes)
•Optional: LangGraph state machine for the diagnose → hypothesize → verify loop

Minimal interface (for the demo)

A terminal dashboard (rich/textual library) that streams live simulated pipeline events and prints the agent's investigation trace and final root-cause report.

8-week timeline

Weeks	Milestone
Wk 1-2	Build synthetic pipeline + failure-injection framework; baseline anomaly detector.
Wk 3-4	Build agent tools (log/schema/metric query) and wire up the diagnose loop.
Wk 5-6	Add hypothesis ranking + remediation-script generation; expand failure-mode library.
Wk 7	Evaluate root-cause accuracy and time-to-diagnosis across 20+ injected incidents.
Wk 8	Polish terminal dashboard, finalize report and incident-replay demo.

Team split (3 members)

- Member 1: Synthetic pipeline + failure injection + anomaly detector
- Member 2: Agentic diagnosis loop + tool design
- Member 3: Evaluation on incident set, root-cause accuracy metrics, terminal UI

Demo deliverable

Trigger a live injected incident → watch the agent investigate in real time in the terminal → it prints a root-cause hypothesis and a remediation diff.

Project 4: Agentic Customer-Support Resolution Engine with Memory

Category: Application — Agentic AI

AI Core / What you're building

An agent that handles multi-turn support tickets end-to-end: retrieves relevant knowledge-base articles, checks a (mock) account/order API, decides whether it can resolve the issue or must escalate, and remembers prior interactions with the same user across sessions (long-term memory).

Why this is industry-grade

Directly modeled on production systems (Intercom Fin, Sierra). Demonstrates RAG + tool-calling + long-term memory + a policy for autonomous escalation decisions — core agentic-AI patterns used in real deployed products.

Key techniques & tools

•RAG over a synthetic knowledge base (FAQ/policies)
•Mock backend tools (order lookup, refund API) — a few Python functions, no real DB needed
•Long-term memory store (vector DB keyed by user id) for cross-session recall
•Escalation-policy module: confidence/uncertainty-based decision to hand off to a human
•Multi-turn dialogue state tracking

Minimal interface (for the demo)

A terminal chat loop simulating a support conversation, with a `--user_id` flag to demonstrate memory persistence across separate CLI sessions.

8-week timeline

Weeks	Milestone
Wk 1-2	Build synthetic KB + mock APIs; basic single-turn RAG-answering agent.
Wk 3-4	Add multi-turn state tracking and long-term memory across sessions.
Wk 5-6	Add escalation policy and confidence estimation; build a labeled test ticket set.
Wk 7	Evaluate resolution rate, escalation precision/recall, memory-recall accuracy.
Wk 8	Polish CLI chat demo, write report with error analysis.

Team split (3 members)

- Member 1: RAG + knowledge base + mock tool APIs
- Member 2: Memory architecture + multi-turn state tracking
- Member 3: Escalation policy, evaluation set, metrics, demo CLI

Demo deliverable

Live two-session terminal chat: session 1 resolves an issue, session 2 (same user) shows the agent recalling context; one ticket intentionally triggers escalation.

Project 5: Multi-Agent Task-Allocation Marketplace (Agentic Coordination)

Category: Application — Agentic AI

AI Core / What you're building

A simulated marketplace of many self-interested LLM agents (e.g., delivery drivers, or compute jobs) that bid on and negotiate task allocations under constraints (time, capacity, cost), with a coordinator agent resolving conflicts, producing an emergent allocation strategy without central hard-coded rules.

Why this is industry-grade

Reflects real industry interest in agentic multi-agent coordination for logistics/scheduling (e.g., warehouse robotics, ride-dispatch). Showcases negotiation protocols, emergent coordination, and game-theoretic evaluation — a differentiated, research-adjacent application.

Key techniques & tools

•Multi-agent framework (AutoGen or a custom message-passing loop)
•Auction/negotiation protocol (English/Vickrey auction or contract-net protocol) driven by LLM bids
•Simulated environment (grid-world or task-graph) with configurable constraints
•Coordinator agent for conflict resolution and re-negotiation
•Baseline comparison vs. a classical optimization solver (e.g., greedy or Hungarian algorithm)

Minimal interface (for the demo)

A terminal-based simulation that prints each negotiation round and renders a simple ASCII/matplotlib snapshot of final task assignments.

8-week timeline

Weeks	Milestone
Wk 1-2	Build simulated environment + task/agent generator; classical-solver baseline.
Wk 3-4	Implement LLM-agent bidding + contract-net negotiation protocol.
Wk 5-6	Add coordinator/re-negotiation logic; scale to 10-20 agents and varied constraints.
Wk 7	Evaluate allocation efficiency vs. baseline, negotiation convergence time, robustness.
Wk 8	Polish visualization, finalize report with efficiency-gap analysis.

Team split (3 members)

- Member 1: Simulated environment + classical baseline solver
- Member 2: LLM negotiation/bidding protocol between agents
- Member 3: Coordinator logic, evaluation metrics, visualization

Demo deliverable

Run a live simulation with 10+ agents negotiating in the terminal, ending in a rendered allocation map and a chart comparing efficiency vs. the classical baseline.

Part B — Research-Style Projects

These projects emphasize a research question, controlled experiments, ablations, and a written analysis suitable for a workshop-paper-style report. Interfaces are reduced to scripts/notebooks that reproduce results.

Project 6: When Does Self-Reflection Help? An Empirical Study of Reflexion-Style Agent Loops

Category: Research — Agentic AI

AI Core / What you're building
A controlled study of self-critique/reflection loops in LLM agents (Reflexion, Self-Refine style): systematically vary task type, number of reflection rounds, and critique prompting strategy to measure when reflection improves task success versus when it causes overthinking, drift, or wasted compute.

Why this is industry-grade
Reflection loops are widely used in industry agents but poorly understood; a clean empirical study with statistically sound ablations is exactly the kind of applied-research contribution valued in both academia and industry AI research teams.

Key techniques & tools

•Agent loop implemented over 2-3 task domains (e.g., code-gen, math word problems, tool-use QA)
•Reflexion/Self-Refine style critique-and-retry mechanism
•Controlled ablations: 0/1/3/5 reflection rounds, critique-prompt variants
•Statistical testing (paired bootstrap or Wilcoxon) over multiple seeds
•Cost/compute-vs-accuracy tradeoff analysis

Minimal interface (for the demo)
A single reproducible script/notebook (`run_experiments.py`) with a results table/plot; no live UI needed, results are the demo.

8-week timeline

Weeks	Milestone
Wk 1-2	Literature review, pick task domains + datasets, implement base agent (no reflection).
Wk 3-4	Implement reflection loop and 2-3 critique-prompt variants.
Wk 5-6	Run full ablation grid across domains/rounds/seeds; collect cost metrics.
Wk 7	Statistical analysis, identify task-dependent patterns, write findings.
Wk 8	Write research report/paper draft, prepare results visualization for defense.

Team split (3 members)

- Member 1: Base agent + task-domain harness for 1-2 domains
- Member 2: Reflection-loop variants + critique prompting
- Member 3: Experiment automation, statistics, cost analysis, and write-up

Demo deliverable
A results notebook run live showing accuracy-vs-reflection-rounds curves per domain, plus significance tests, framed as a mini research paper.

Project 7: Emergent Communication Efficiency in Multi-Agent LLM Systems

Category: Research — Agentic AI

AI Core / What you're building

Study how multiple LLM agents communicating to solve a shared task develop (or fail to develop) efficient, compressed communication protocols when message length/token budget is constrained, compared to unconstrained natural-language communication.

Why this is industry-grade

Connects classic emergent-communication research (traditionally done with RL agents) to modern LLM-based multi-agent systems — a genuinely novel angle with clear research contribution and publishable framing.

Key techniques & tools

•Multi-agent cooperative task (e.g., referential game, collaborative puzzle-solving)
•Token-budget-constrained message passing between agents
•Comparison: free-form NL messages vs. budget-constrained vs. learned-compressed protocol prompting
•Metrics: task success rate, message entropy/compression ratio, protocol stability over rounds
•Qualitative analysis of emergent shorthand/protocol patterns

Minimal interface (for the demo)

A script that runs agent-pair episodes and logs transcripts + metrics to a CSV; results shown via notebook plots, no persistent UI required.

8-week timeline

Weeks	Milestone
Wk 1-2	Design cooperative task + baseline unconstrained-communication agent pair.
Wk 3-4	Implement token-budget constraint and compression-prompting condition.
Wk 5-6	Run large-scale episodes across conditions/budgets; log transcripts for analysis.
Wk 7	Quantitative + qualitative analysis of emergent protocols; entropy/compression metrics.
Wk 8	Write findings, prepare transcript examples and result plots for presentation.

Team split (3 members)

- Member 1: Task environment + baseline agent-pair implementation
- Member 2: Budget-constrained/compression conditions + logging pipeline
- Member 3: Metrics (entropy, success rate), qualitative analysis, write-up

Demo deliverable

Live run of a few episodes showing transcripts under different budgets, followed by summary plots of success-rate vs. communication-budget.

Project 8: A Benchmark for Tool-Use Robustness and Hallucinated Tool Calls in Agentic Pipelines

Category: Research — Agentic AI

AI Core / What you're building
Build a benchmark of tasks with deliberately noisy/ambiguous/adversarial tool specs (misleading names, missing parameters, flaky/erroring tools) and measure how often agents hallucinate tool calls, misuse tools, or fail to recover — then propose and test a mitigation (tool-schema validation layer, verification step, or uncertainty-gated calling).

Why this is industry-grade
Tool-use robustness/hallucination is an open, actively-discussed problem in agentic AI (cited by Anthropic, OpenAI, and academic surveys); a well-built benchmark plus a mitigation study is a strong, citable research contribution.

Key techniques & tools
•Curated/synthetic tool-use task suite with adversarial tool specs
•Multiple agent frameworks/prompting strategies tested against the suite (for generality)
•Mitigation techniques: schema validation, self-verification before calling, confidence gating
•Metrics: hallucinated-call rate, recovery rate after a failed/erroring tool call, task success rate
•Error taxonomy (categorize failure modes qualitatively)

Minimal interface (for the demo)
A benchmark-runner script producing a results table/leaderboard (CSV/Markdown) across conditions; no UI needed beyond the printed leaderboard.

8-week timeline

Weeks	Milestone
Wk 1-2	Design task suite + adversarial tool specs; build base agent harness.
Wk 3-4	Run baseline evaluation across models/frameworks; build error taxonomy.
Wk 5-6	Implement and test 2-3 mitigation techniques against the benchmark.
Wk 7	Full comparative evaluation, statistical significance of mitigation gains.
Wk 8	Write benchmark paper-style report; package benchmark for reuse.

Team split (3 members)
- Member 1: Task suite + adversarial tool-spec design
- Member 2: Mitigation techniques (validation/verification/confidence gating)
- Member 3: Evaluation harness, metrics, error taxonomy, leaderboard/report

Demo deliverable
Live run of the benchmark leaderboard script showing hallucination/recovery rates with and without the proposed mitigation.

Project 9: Long-Horizon Planning in LLM Agents: ReAct vs. Tree-of-Thought vs. Graph-of-Thought

Category: Research — Agentic AI

AI Core / What you're building

A rigorous head-to-head empirical comparison of three planning/reasoning strategies for LLM agents on long-horizon, multi-step tasks (e.g., multi-hop QA, planning puzzles, multi-step tool pipelines), analyzing success rate, token/cost efficiency, and failure modes (getting stuck, looping, premature termination) as horizon length increases.

Why this is industry-grade

Planning-strategy choice is a central, unresolved design decision in agentic systems; a controlled comparison with cost-adjusted metrics (not just accuracy) is directly useful to practitioners and is a well-scoped, publishable empirical study.

Key techniques & tools

•Implementations of ReAct, Tree-of-Thought, and Graph-of-Thought over a shared agent scaffold
•Long-horizon task suite with controllable difficulty/horizon length
•Cost-normalized metrics (success per token/dollar), not just raw accuracy
•Failure-mode taxonomy (loop detection, drift, premature stop) via transcript analysis
•Statistical testing across multiple seeds/models

Minimal interface (for the demo)

A single experiment-runner script/notebook producing comparison tables and plots; results are the entire demo.

8-week timeline

Weeks	Milestone
Wk 1-2	Implement shared agent scaffold + one strategy (ReAct) as baseline; pick task suite.
Wk 3-4	Implement Tree-of-Thought and Graph-of-Thought variants on the same scaffold.
Wk 5-6	Run full comparison across horizon lengths/models/seeds; collect cost data.
Wk 7	Failure-mode analysis, cost-normalized ranking, statistical tests.
Wk 8	Write report with recommendations on when to use which strategy.

Team split (3 members)

- Member 1: Shared scaffold + ReAct + Tree-of-Thought implementation
- Member 2: Graph-of-Thought implementation + task-suite difficulty scaling
- Member 3: Cost/metric analysis, failure taxonomy, statistical write-up

Demo deliverable

Live notebook run comparing all three strategies on a fresh task instance, ending in a summary chart of success-vs-cost across horizon lengths.

Project 10: Memory Architectures for Long-Horizon Agents: Vector Recall vs. Episodic Summarization vs. Hybrid

Category: Research — Agentic AI

AI Core / What you're building
Compare three memory architectures for LLM agents operating over long interaction histories — raw vector-similarity retrieval, periodic episodic summarization, and a hybrid approach — measuring recall accuracy, consistency over time, and degradation as history length grows on long-running agentic tasks.

Why this is industry-grade
Memory is one of the biggest open problems limiting real-world agent deployments (customer support, personal assistants); a controlled comparison with degradation curves is a concrete, industry-relevant research contribution.

Key techniques & tools
•Vector-store memory (embedding + similarity search) baseline
•Episodic/periodic summarization memory (LLM-generated running summaries)
•Hybrid memory combining both, with a router deciding which to query
•Long-horizon synthetic conversation/task generator with injected recall-probe questions
•Metrics: recall accuracy vs. history length, contradiction rate, latency/cost per memory type

Minimal interface (for the demo)
A script that simulates long conversations and periodically probes memory recall, printing a results table; notebook plots for the write-up.

8-week timeline

Weeks	Milestone
Wk 1-2	Build long-horizon conversation/task generator with recall-probe injection.
Wk 3-4	Implement vector-memory and summarization-memory baselines.
Wk 5-6	Implement hybrid memory + router; run full comparison across history lengths.
Wk 7	Analyze degradation curves, contradiction rates, cost tradeoffs.
Wk 8	Write report with recommendations and a proposed hybrid design summary.

Team split (3 members)
- Member 1: Conversation/task generator + recall-probe design
- Member 2: Vector-memory + summarization-memory implementations
- Member 3: Hybrid memory/router, evaluation metrics, degradation-curve analysis

Demo deliverable
Live run showing a simulated long conversation with periodic recall probes, ending in a chart of recall accuracy vs. conversation length for all three architectures.

General Notes for the Team

- Pick one project by end of Week 1 — do not split effort across ideas.
- For every project, keep a single shared repo with a `/experiments` or `/agents` folder as the core artifact; the interface (CLI/notebook/single-page app) should take no more than ~10% of total engineering time.

• Log everything (prompts, tool calls, model outputs) from day one — this log is what turns into your evaluation tables and demo transcripts later.

• Budget Week 7 explicitly for evaluation/ablations and Week 8 for writing and demo polish; do not leave evaluation to the last week.

• For research-style projects, treat the final report as a short workshop-paper draft (Abstract, Related Work, Method, Experiments, Results, Discussion) — this structure alone earns significant grading credit.